完整会话记录 / Complete Session Dialogue

> Session: ses_e5ada32afffd7ucLkj9QlbZEe0 · STAT6207期末作业项目
> 完整对话原文，不是摘要。工具调用日志见完整版 Markdown/HTML。
> Verbatim dialogue, not a summary. Tool logs are in the full Markdown/HTML version.

用户 / User • 1

如题，我们老师要求我们完成一项期末小组作业，目前需要提交一份单页的pdf计划书。你能否先读取一下里面的内容，告诉我要如何实施计划以完成项目？

助手 / Assistant • 16

我已读完 `AI_CV_Matching_Project_Plan.pdf`（以及中文对照版），并把随附的 `Manatal Examples - JD & CV (anonymised).xlsx`、100 份 CV 和 10 份 JD 也盘点了一遍。先说结论：

> 这个项目的核心在于：用约 50 组可信的人工标注，证明一个**简单、可解释的匹配器**能稳定把合适的人排进 Top-5，并且给出的理由站得住脚；做出“很准的模型”并不是核心目标。单页计划书要让人一眼看到四件事：**数据怎么来、标签怎么打、基线怎么评、风险怎么控。**

一、任务要求（从 PDF 提炼）

输入：一份 JD
输出：排好序的 Top-5 CV，每份附一句简短理由
验收：有经验的 Second Talent 招聘顾问会认可这份短名单

三项交付物：

1. **准备数据**：匿名化技术岗 JD/CV + 约 50 组招聘顾问认可的 good-match 配对
2. **写匹配指南**：从 JD/CV 分别抽什么、每个因素占多大权重
3. **搭基线**：一个简单 matcher + 一套测试，说明它多大程度能选对人

范围边界：技术岗优先；不做上线 HRIS 功能、不碰未脱敏数据、不买/爬 LinkedIn、不做销售岗。

二、现有数据盘点（这直接决定计划怎么写）

数据	实际情况	对计划的影响
JD 表	50 份，其中 Engineering 16 + Data & AI 7 = **23 份技术岗**	离目标 100–200 份差 **77–177 份**
CV 表	100 份，其中 Engineering 30 + Data & AI 16 = **46 份技术岗**	技术 CV 够做 pilot，但不够覆盖大量 JD
匹配标签	**完全没有**	50 组 good-match 必须自己标
JD PDF	`JD + Hired CV examples` 里只有 **10 份 JD**（P01–P10）	而且多为财务、销售、市场、产品、设计等非技术岗
Hired CV	该文件夹里**没有 CV**，只有 JD	不能直接拿来当 50 组正样本，需要向老师/Eric 确认

数据	实际情况	对计划的影响
CV 字段	职位、公司、技能、工作经历、教育都有，PDF 格式统一	抽取难度低，适合做结构化 baseline
缺失字段	很多 CV 没有期望薪资、notice period、语言、地点	评分时这些应记为 **unknown（中性）**，不能当 0 分

最重要的发现：现有数据只能支撑一个 **23 个技术 JD 的 pilot**，而 brief 写的是 100–200 个 JD。单页计划书里必须明确写“数据补充计划”或“本期 scope 缩小到现有 23 个 JD + 扩展方案”，不要假装差距不存在。

三、实施路线：6 个工作包

WP0 项目启动与口径确认（第 1 周初）

- 和老师/Eric 确认四件事：
  - “good match” 的 operational 定义：是“值得进 shortlist”，还是“完美匹配”？
  - 本期是用现有 23 个技术 JD，还是要从 Manatal 补到 100+？
  - 是否允许调用 Claude API？数据脱敏到什么程度？
  - 最终验收指标（Top-5 hit rate？招聘顾问盲评一致率？）
- 先锁定 **2–3 个技术岗位族**，例如 AI/ML、Backend、Full-stack，不要一上来铺 8 个族。
- 产出：1 页 scope + 成功指标。

WP1 数据工程与隐私（第 1 周）

- 建 `jd_id` / `cv_id` 主表，把 xlsx 和 PDF 统一解析成 JSONL。
- 清洗：去重、统一地点（KL / Kuala Lumpur）、统一职位名、建技能词典（Python、Go、React、Kubernetes…）、计算相关年限。
- 脱敏复核：姓名、邮箱、电话、地址、照片、LinkedIn URL；只把脱敏副本送外部 API；人工抽查 20 份；**Eric 批准后才外传。**
- 产出：`jds.jsonl`、`cvs.jsonl`、`data_dictionary.md`、masking checklist。

WP2 写匹配指南（第 1–2 周）

定义两套 schema：

JD 抽取字段：must-have skills、nice-to-have、min years、seniority、location/remote、salary budget、languages、notice period、education、domain。

CV 抽取字段：skills + 证据、每项技能年限、总相关年限、seniority、location、expected salary、notice、languages、education、tenure/gaps。

权重直接映射 brief 的四档：

档位	因素	建议分值
Highest	硬技能重合（must/nice）、相关年限	35 + 20 = **55**
High	资历层级、期望薪资 vs 预算	15 + 10 = **25**
Medium	行业/领域、稳定性、学历/证书	10 + 5 + 5 = **20**
Filters	地点/时区/远程、notice、语言	**pass/fail 门槛**

标注用 0–3 分 rubric：

- **3** = 强烈推荐面试

- **2** = 可以进 shortlist
- **1** = 边缘，可看可不看
- **0** = 不合适

产出：2–3 页 matching guide + 一页标注手册（含正例/负例示例）。

WP3 标注约 50 组配对（第 2 周）

- 采样方式很重要：**12–15 个技术 JD × 4–5 份 CV = 50–75 对。**
- 每个 JD 至少包含：1 个强正例、1 个边缘例、2 个明显负例（hard negatives：技能差一点、seniority 过高/过低、地点或薪资不符、非技术背景）。
- **不要随机抽 CV**，否则几乎全是负例，模型学不到边界。
- 前 20 对由两人独立标注，算 Cohen's kappa；分歧讨论后定稿。
- 每条标签必须写理由，并引用 CV 里的证据（技能、年限、公司、项目）。
- 产出：`gold_pairs.csv`（`jd_id, cv_id, label, reason, annotator`）。

WP4 搭基线模型（第 2–3 周）

按“先跑通、再增强”的顺序：

- **v0**：硬过滤 + 关键词重合，先跑通流程。
- **v1（主基线）**：技能词典 + BM25/TF-IDF + 加权评分。
- **v2**：embedding 语义相似度（sentence-transformers 或多语言 embedding），hybrid 分数。
- **v3（可选）**：Logistic Regression / LightGBM 学习排序。**50 组样本太少，只能做 ablation，不能当主模型。**

评分公式建议：

```
score = 0.35 * 技能重合
      + 0.20 * 相关年限
      + 0.15 * 资历层级
      + 0.10 * 薪资匹配
      + 0.10 * 行业/领域
      + 0.05 * 稳定性
      + 0.05 * 学历/证书
```

缺失字段记 **0.5（中性）**，不要记 0，否则会系统性惩罚匿名化简历。

理由生成：**模板优先**，例如“匹配 5/6 项必备技能；6 年 Python；缺 Kubernetes；地点匹配”。LLM 只负责润色，禁止无证据陈述。

产出：`match.py`，接口 `rank(jd_id) -> top5 + reasons`。

WP5 评估（第 3 周）

- 主指标：**Top-5 hit rate**（Top-5 里是否至少有一个 good match）、**nDCG@5**。
- 次指标：Precision@5、MRR、filter precision。
- 切分：**按 JD 做 leave-one-JD-out 或 5-fold group split**，不要随机按 pair 切，否则同一 JD 会泄漏。
- 对照：random、keyword-only、embedding-only。
- 人工评估：3 位 reviewer 盲评 5 个 JD 的 Top-5，1–5 分，报告一致率。
- 抽取质量：人工核对 20 份文档，报告 skill F1、years MAE、seniority accuracy。
- 建议目标（和老师确认后再写死）：

    - Top-5 hit rate ≥ 70%
    - nDCG@5 ≥ 0.75
    - 比 keyword baseline 相对提升 ≥ 15%
    - 抽取 F1 ≥ 0.85
    - 招聘顾问盲评一致率 ≥ 80%
- 产出：evaluation report + error taxonomy（解析错误、漏 must-have、seniority 错、地点/薪资错、理由幻觉）。

WP6 交付与答辩（第 4 周）

- 代码 repo + README + demo notebook。
- 5–8 页报告。
- 10 分钟 demo：现场选 1 个 JD，展示 Top-5 和理由。
- 单页 plan PDF。
- 备用方案：如果 JD 数量不够，展示“23 个 JD pilot + 扩展到 100+ 的具体计划”。

四、四周排期与分工

周	主要任务	里程碑
Week 1	kickoff、数据索引、脱敏复核、schema、guide v1、标 10 对	数据可用 + guide 定稿
Week 2	标完 50 对 + kappa、v0/v1、评估脚本	主基线可跑
Week 3	v2、完整评估、error analysis、理由生成、盲评	指标和错误分析出来
Week 4	报告、demo、单页 PDF、buffer	全部交付

4–5 人分工建议：

- **A 数据/隐私**：解析、清洗、脱敏、数据字典
- **B 标注/指南**：matching guide、标注手册、标签 QA
- **C 建模**：v0/v1/v2、评分与理由
- **D 评估**：指标、切分、盲评、错误分析
- **E 项目/写作**：排期、单页计划书、报告、demo（可由 A–D 兼任）

五、单页 PDF 计划书建议结构

一页 A4，建议 3 栏或上下结构，正文控制在 **800–1000 中文字**（英文约 500–600 词），字号 9–10pt，配一张 5 步流程图：

1. **标题 + 团队/日期/客户**
2. **问题与目标**（2–3 行）：招聘顾问人工筛简历慢；目标 JD → Top-5 + 理由
3. **范围与数据**（3 行）：50 JD / 100 CV，其中 23 个技术 JD；50 组标签；明确 out of scope
4. **方法**（核心区域）：5 步流程图 + 权重表（Highest/High/Medium/Filters）
5. **评估与成功标准**：Top-5 hit rate、nDCG@5、抽取 F1、盲评一致率
6. **4 周时间线**：用 4 个横条或里程碑
7. **风险与应对**：隐私、标签少、JD 数量不足、字段缺失
8. **交付物**：数据包、guide、baseline repo、评估报告、demo

单页计划书不要写成“技术方案书”，老师更想看到：**目标可衡量、数据有来源、标签有方法、风险有预案、四周能做完。**

六、最容易被问倒的点

1. **23 个技术 JD vs 目标 100–200**：必须写明补充计划或缩小 scope，不能回避。
2. **50 组样本太少，不能训练复杂模型**：定位为“评估集 + 规则/embedding 基线”，ML 只做 ablation。
3. **标签主观**：双人标注 + rubric + 分歧仲裁，报告 kappa。
4. **匿名 CV 缺薪资/notice**：unknown 按中性处理，不惩罚。
5. **LLM 抽取幻觉**：固定 schema + 证据 span + 20 份人工核对。
6. **数据泄漏**：按 JD 切分，不按 pair 随机切。
7. **隐私合规**：脱敏副本 + 人工复核 + Eric 批准，才允许离开 Second Talent。
8. **“招聘顾问会认可”太虚**：把它变成可测指标（Top-5 hit rate + 盲评一致率）。

如果你愿意，我可以直接帮你做下一步，三选一：

1. **起草这份单页 PDF 计划书**（中英文版，按上面的结构排版）
2. **写数据索引/解析脚本**（把 xlsx + 100 份 CV PDF 变成 `jds.jsonl` / `cvs.jsonl`）
3. **起草 matching guide + 标注表模板**（含权重、0–3 rubric、正负例）

你希望先做哪一个?

用户 / User • 17

你要不先写一份计划书？先不需要对数据进行处理，先把计划书pdf写出来

助手 / Assistant • 19

好的。我先把单页计划书做成 PDF（中文版，带少量英文术语），不处理原始数据。做之前我会先把版式排成严格 A4 单页，再渲染检查是否溢出。

助手 / Assistant • 85

计划书已经写好，并且按 **A4 单页**排版、渲染检查过：中文版和英文版都确认是 **1 页。**

文件位置：

- 中文版：`AI_CV_Matching_OnePage_Plan_CN.pdf`
- 英文版：`AI_CV_Matching_OnePage_Plan_EN.pdf`
- 可编辑源文件：`plan_source/one_page_plan_cn.html`、`plan_source/one_page_plan_en.html`
- 重新导出说明：`plan_source/README.md`

中文版已在预览窗口打开。计划书包含 7 个部分：

1. **问题与目标**：从 JD 到 Top-5 + 逐条理由，并给出可量化指标
2. **方法与权重**：5 步流程 + Highest / High / Medium / Filters 四档权重
3. **评估方案**：50 组正例 + 50 组 hard negatives、0–3 rubric、双人标注与 kappa、按 JD 切分
4. **数据与范围**：如实写出 23 个技术 JD 与 100–200 目标的缺口，并给出 pilot + 补充方案
5. **四周排期**：W1 数据与指南 → W2 标注与基线 → W3 增强与评估 → W4 交付
6. **风险与应对**：JD 不足、样本少、字段缺失、标签主观、隐私合规

7. **交付物**：数据包、Matching Guide、Python repo、评估报告、答辩 demo

有两点建议你提交前确认：

- PDF 里的 **Top-5 ≥70%、nDCG@5 ≥0.75、盲评一致率 ≥80%** 是我按合理基线设的“建议目标值”，最好和老师或 Eric 确认后再定稿。
- 页脚留了 **“小组 / 成员”** 空位。把组名和成员名单发我，我可以直接填好并重新导出；也可以按需改配色、加学校 logo，或改成中英对照版。

用户 / User • 86

稍等一下，你注意到工作目录下的这个文件了吗？在写计划书的时候是否将其考虑了进去 Attached file: `D:\softs_usual\opencode_windows\stat6207_final_groupproject\Manatal Examples - JD & CV (anonymised).xlsx`

助手 / Assistant • 122

注意到了，而且计划书里的数据口径就是从这里来的。不过你提醒得对——我一开始漏看了这份材料配套 PDF 的后半部分，已经修正并重新导出了。

1. xlsx 里我用了什么

`Manatal Examples - JD & CV (anonymised).xlsx` 我在写计划书前已经读过：

- `JD Examples`：**50 份 JD**，其中 Engineering 16 + Data & AI 7 = **23 份技术岗**
- `CV Examples`：**100 份 CV**，其中 Engineering 30 + Data & AI 16 = **46 份技术岗**
- 没有任何 match label
- CV 表已有 `Current title / Company / Skills / Work history / Education` 等结构化字段

这些数字都进了计划书的 KPI、数据表和 scope 部分，数据表里也直接写了这个文件名。

2. 我之前漏掉、现在已修正的一点

`JD + Hired CV examples` 里的 10 份 PDF，我最初只读了第 1 页，看到是 JD，就在计划书里写成了“仅含 JD”。实际上每份 PDF 后面都有：

> `CV OF THE HIRED CANDIDATE`

也就是说，这 10 份 PDF 是 **10 组 JD + hired CV 的招聘顾问认可正例**，不是 0 组标签。这个对计划影响很大，已经改到两个版本里：

- 数据表新增：**10 份 PDF = 10 组 JD + hired CV 正例**
- 匹配标签：**已有 10 组正例 → 目标 50 正例 + 50 负例**
- Gold set：**10 组 hired 正例（校准）+ 40 组技术岗正例 + 50 组 hard negatives ≈ 100 对**
- W1：复核 10 组 hired 正例，校准 rubric
- W2：补标 40 组技术正例 + 50 组负例

3. 已重新导出并检查

- `AI_CV_Matching_OnePage_Plan_CN.pdf`（1 页，已更新）

- `AI_CV_Matching_OnePage_Plan_EN.pdf`（1 页，已更新）
- 源文件：`plan_source/one_page_plan_cn.html`、`plan_source/one_page_plan_en.html`
- 说明：`plan_source/README.md`

4. 有一个口径需要你定

现在计划书按 **“10 组 hired 正例（校准）+ 40 组技术正例” = 50 组正例** 来写。另一种更严格的做法是：

- 10 组 hired 只做校准，不计入技术岗指标
- 另外标 **50 组纯技术岗正例** + 50 组负例（总量约 110 对）

如果老师更看重“技术岗 50 组正例”的纯度，我可以改成后一种；如果希望四周内稳妥完成，现在这版更现实。你倾向哪一种?

用户 / User • 123

准确说我检查了一下发现，Anonymised-CVs-100文件夹里是100个匿名的CV（纯简历，且为原始文件形式）； 而Manatal Examples – JD & CV (anonymised)这个excel起始表格里则是记录了50个JD工作描述和100个CV（概括整理的标签形式，且这100个cv就是对应Anonymised-CVs-100文件夹里的每个实体cv），但没有任何匹配标签； JD + Hired CV examples文件夹则是一个“优秀匹配范例”文件夹，也就是工作描述与 被雇佣者CV 完美匹配的实例记录

助手 / Assistant • 158

明白了，这个区分很关键。最新版计划书已经按你说的三层数据结构重写，并且两个 PDF 都重新导出、确认仍是 **A4 单页**。

现在计划书里怎么用这三部分数据

1. `Anonymised-CVs-100` **= 100 份原始 CV PDF**

作为原始证据来源：抽取技能、年限、项目经历时以 PDF 为准，理由也要能引用 PDF 里的证据。

2. `Manatal Examples - JD & CV (anonymised).xlsx` **= 50 JD + 100 条结构化 CV 摘要**

- 这 100 条摘要与 100 份 PDF 按 `Ref`（CV-001…CV-100）一一对应
- 表里没有匹配标签
- 计划书把它当作 **structured index / silver label**：用摘要做初始字段和索引，再用 LLM 在 PDF 上补证据、补缺失字段，并与摘要交叉核对，而不是把 xlsx 和 PDF 当成两套独立数据

3. `JD + Hired CV examples` **= 10 组“完美匹配”范例**

- 每份 PDF = 1 份 JD + 1 份 hired CV
- 计划书把这 10 组作为 **known-good 测试集**：给定 hired JD，看系统能不能把实际被雇佣的 CV 排进 Top-1 / Top-5
- 它们不参与调参，避免“自己考自己”

计划书具体改了什么

- 数据表：

    - 起始表：**50 JD / 100 CV 摘要（技术 23 / 46）**
    - Hired 范例：**10 组 JD + hired CV 完美匹配**
    - 匹配标签：**10 组 known-good → 目标 50 正例 + 50 负例**
    - 数据文件：**Manatal…xlsx + 100 份 CV PDF**
- 评估：
    - **10 组 known-good** 做 Top-1 / Top-5 检索测试
    - 另标 **50 组技术岗正例 + 50 组 hard negatives**，用于 Top-5 hit rate、nDCG@5 等主指标
- 排期：
    - W1：按 `Ref` 对齐 100 份 PDF，复核 10 组 hired
    - W2：标 50 技术正例 + 50 负例 + kappa

文件

- `AI_CV_Matching_OnePage_Plan_CN.pdf`（已更新，预览窗口打开的就是这版）
- `AI_CV_Matching_OnePage_Plan_EN.pdf`
- 源文件：`plan_source/one_page_plan_cn.html`、`plan_source/one_page_plan_en.html`
- 数据口径说明：`plan_source/README.md`

一个方法上的提醒：10 组 known-good 样本量很小，适合做 **sanity check / 回归测试**，不适合单独支撑统计结论；主评估仍然是 50+50 的 gold set。测试时把 10 份 hired CV 加入候选池、单独报告结果，不混进标注集。

如果时间紧，50 正例 + 50 负例的工作量偏大，可以降到 40+40，但指标置信度会低一些。你希望保持现在的 50+50，还是改成更稳妥的 40+40?

用户 / User • 159

我认为，这10组已知的优秀配对，实际上只是一个例子，虽然有一定参考作用但主要还是应当依靠原始数据集（起始表格与100分匿名CV），以及设计好合适的打分规则来产出“优质配对组”

用户 / User • 193

你好了没有？为什么这么慢

助手 / Assistant • 195

抱歉久等。主要卡在两件事：按你的意见重写了数据逻辑，以及旧的中文 PDF 被预览窗口占用、无法覆盖，所以新版另存了文件名。

新版文件：

- 中文最终版：`AI_CV_Matching_OnePage_Plan_CN_final.pdf`
- 英文版：`AI_CV_Matching_OnePage_Plan_EN.pdf`
- 旧的 `AI_CV_Matching_OnePage_Plan_CN.pdf` 是过期版本，关闭预览后可删掉

这版按你的意见改成了：

- **10 组 hired 只作参考案例**，只用来校准 rubric，不计入标签、不作为主评估

- **主数据是 50 JD + 100 CV（xlsx 摘要与 PDF 按 Ref 对应）**
- **先用打分规则生成候选配对**，再由招聘顾问按 rubric 独立确认 **50 组技术正例 + 50 组负例**
- 评估按 JD 留出，避免“规则自己生成、自己考自己”的自证问题
- 风险栏新增：**规则自证 / 标签主观 → 独立确认 + 按 JD 留出 + 双人标注 + kappa**

两份都是 A4 单页。中英文源文件在 `plan_source/`，组名和成员还留着空位，把名字发我可以直接填好。

系统 / System • 196

Today's date is now: Sat Nov 07 2026

用户 / User • 197

现在先不管，我现在有了一个新的任务。虽然是基于本数据集但是任务相对来说比较独立，所以我希望你将所有“工作期间所产生的文件”保存到目录：D:\softs_usual\opencode_windows\stat6207_final_groupproject\HW4_6207下（你可以将原始数据集复制一份到这个文件夹里，然后在这个新的目录开始工作），在这个新的目录进行工作

以下是作业的具体要求： 1使用本地大语言模型（LLM）从简历（CV）和职位描述（JD）中提取关键词 2合并/聚类相似的关键词 3构建自编码器（AE）模型对简历和职位描述进行编码 4针对基准数据（Ground Truth）中的每个职位描述，检索出排名前 10 的简历 5评估检索的召回率（Recall） 6进行错误分析以寻求改进方案 7将实验结果整理成pdf中英双语的报告文件 我认为关于步骤1，本地大语言模型你可以下载qwen系列的然后调用，或者试着看一下lm studio内是否具有类似的本地LLM文件；此外，关于”从CV和JD中提取关键词“，我认为这里的CV，指的是D:\softs_usual\opencode_windows\stat6207_final_groupproject\Anonymised-CVs-100\Anonymised CVs 文件夹下的简历合集；而JD则是指Manatal Examples – JD & CV (anonymised) 表格里的各种职位描述。作为示例的JD Hired CV examples文件夹可以先不管

在开始工作之前，你先告诉我你要如何开展工作?

助手 / Assistant • 198

我先做一次只读环境检查（不创建文件），确认本地 LLM、GPU 和依赖情况，然后给你一份可执行的方案；其中有一个关键点需要你确认：**Ground Truth 从哪里来。**

助手 / Assistant • 200

可以，这个任务我能做。我先检查了环境，下面是我的完整开展方案。**我先不创建/复制任何文件**，等你确认几个关键点后再在 `HW4_6207` 开工。

0. 环境检查结果

- GPU：RTX 5080 Laptop，16GB 显存
- LM Studio 已安装，已有 3 个本地模型：
  - `qwen2.5-coder-14b-instruct`（14B，Qwen2）
  - `deepseek/deepseek-r1-0528-qwen3-8b`（8B，Qwen3）
  - `text-embedding-nomic-embed-text-v1.5`（embedding）

- LM Studio 显示 Server ON，但 `localhost:1234` 暂时拒绝连接；我会先重载模型/重启 server 再批量跑
- Python 已有 `torch`（CPU 版）、`sklearn`、`pandas`、`pymupdf`、`openpyxl`、`reportlab`；缺 `sentence-transformers`、`faiss`
- AE 数据量很小（约 150 份文档），CPU 足够；LLM 走 LM Studio 的 GPU，不依赖 CUDA 版 PyTorch
- `HW4_6207` 目录已存在但目前是空的

1. 最关键的前提：Ground Truth 从哪里来

这是开始前必须确认的。现有数据里：

- `Anonymised-CVs-100`：100 份 CV PDF，**没有 JD 匹配标签**
- `Manatal Examples...xlsx`：50 份 JD + 100 条 CV 摘要，**没有匹配标签**
- `JD + Hired CV examples`：10 组 JD + hired CV，是唯一带“完美匹配”标签的数据

我核对过：这 10 份 hired CV 基本不在那 100 份 CV 里（例如 P01 的 Cardia、P04 的 BP 都没有出现在 xlsx 的 100 条记录中）。所以：

> 如果 Ground Truth 用这 10 组，候选池必须是 **100 份 CV + 10 份 hired CV = 110 份**；否则正确 CV 根本不在池子里，Recall 必然是 0。

我的建议是：

- **把这 10 组只用于评估**，不参与关键词提取、不参与 AE 训练
- 每组 JD 对应 1 份相关 CV，计算 **Recall@10 / Recall@5 / MRR**
- 你说的“先不管这个文件夹”，如果是指“不作为输入数据”，那和这个方案兼容；如果你希望连评估也不用它，就需要你另外提供一份 Ground Truth 标签文件，否则 Recall 无法计算

这一点需要你确认。

2. 工作目录设计

所有产出都放在：

```
HW4_6207/
├── data/
│   ├── raw/                # 复制进来的 100 份 CV PDF + xlsx
│   ├── processed/          # cv_text.jsonl, jd_text.jsonl
│   └── ground_truth/       # 10 组 GT 的 jd_id/cv_id（如确认使用）
├── src/
│   ├── parse_data.py
│   ├── extract_keywords.py
│   ├── cluster_keywords.py
│   ├── train_ae.py
│   ├── retrieve.py
│   ├── evaluate.py
│   └── error_analysis.py
├── outputs/
│   ├── keywords/           # cv_keywords.jsonl, jd_keywords.jsonl
│   ├── clusters/           # keyword_clusters.json, vocab.json
│   ├── models/             # ae_model.pt, scaler.pkl
│   ├── retrieval/          # rankings.csv
│   └── metrics/            # metrics.json, per_query.csv
├── report/
```

```
|   ├── report_cn_en.html
|   ├── figures/
|   └── HW4_Report_CN_EN.pdf
├── README.md
└── requirements.txt
```

3. 技术路线（对应作业 1–7）

Step 1：本地 LLM 关键词提取

- 数据：
  - CV：`Anonymised-CVs-100\Anonymised CVs` 下 100 份 PDF
  - JD：xlsx 的 `JD Examples` 表，`Job description` 列，共 50 份
  - 不把 `JD + Hired CV examples` 作为提取输入
- 调用方式：LM Studio 的 OpenAI-compatible API，本地模型，temperature=0
- 先在同一批 5 份文档上对比：
  - `qwen2.5-coder-14b-instruct`
  - `deepseek-r1-0528-qwen3-8b`
- 选 JSON 合规率、关键词质量更好的做主力；都不理想再下载 `Qwen2.5-7B-Instruct`
- 输出 schema：
  - JD：`must_have_skills`、`nice_to_have_skills`、`tools`、`domains`、`seniority`、`min_years`、`languages`、`location`、`certifications`
  - CV：`skills`、`tools`、`domains`、`roles`、`years`、`seniority`、`education`、`certifications`、`languages`、`location`
  - 每个关键词带 `term`、`category`、`importance`（1–3）、`evidence`
- 所有结果缓存为 JSONL，按文档 hash 去重，失败自动重试
- 质量检查：人工抽查 10 份；JSON 合法率；CV 关键词与 xlsx `Skills` 列交叉核对

Step 2：合并/聚类相似关键词

- 归一化：小写、去标点、单复数、缩写（JS→JavaScript、ML→Machine Learning 等）
- 用本地 `nomic-embed-text` 对关键词做 embedding
- 用 cosine + 层次聚类/社区发现合并同义词
- 每个 cluster 选一个 canonical term（最高频或 LLM 决定）
- 输出：
  - `keyword_clusters.json`
  - `vocab.json`
  - 文档–关键词矩阵（TF-IDF 或 binary）
- 人工检查 top 50 clusters，避免把“Java”和“JavaScript”这类错误合并

Step 3：构建 Autoencoder（AE）

- 输入：文档–关键词矩阵（约 150 份文档 × 词表维度）
- 结构：`V → 512 → 128 → bottleneck 32/64 → 128 → 512 → V`
- GELU + Dropout + BatchNorm；可选 denoising（随机 mask 15% 关键词）
- 损失：MSE / BCE；AdamW；early stopping
- 输出：
  - `ae_model.pt`
  - 所有 CV/JD 的 32/64 维向量 `doc_vectors.npz`
- 对比基线：

    - 原始 TF-IDF cosine
    - BM25（如可用）
    - 关键词重叠率
    - Nomic embedding cosine
- AE 只做无监督编码，不使用 Ground Truth 标签

Step 4：Top-10 检索

- 对 Ground Truth 中每个 JD：
    - 用 AE 编码 JD
    - 与候选池所有 CV 向量算 cosine similarity
    - 返回 Top-10
- 候选池：
    - 如果 GT 用 10 组 hired：100 + 10 = 110 份 CV
    - 如果 GT 是另外的标签文件：按你提供的范围
- 保存每个 JD 的 Top-10 排名、分数、关键词重叠解释

Step 5：Recall 评估

- 主指标：**Recall@10**
- 辅助：Recall@5、MRR@10、nDCG@10
- 如果每个 JD 只有 1 份相关 CV，Recall@10 就是“命中率”；报告：
    - 总体均值
    - 每个 query 的结果
    - Wilson 95% 置信区间（样本只有 10 个 query 时很重要）
- 同时对比 AE / TF-IDF / BM25 / Nomic embedding / 关键词重叠

Step 6：错误分析

对每个未命中的 query 分类：

1. 关键词抽取错误/漏抽
2. 同义词合并错误
3. must-have 权重不够
4. seniority / location / language 不匹配
5. AE bottleneck 丢失信息
6. Ground Truth 本身有歧义

再做消融：

- 有/无关键词聚类
- bottleneck 16/32/64/128
- 有/无 denoising
- LLM 关键词 vs 原始文本 TF-IDF
- AE vs 各 baseline

最后给出 3–5 条可执行的改进建议。

Step 7：中英双语 PDF 报告

建议 **一份 PDF，中英双语对照/分节**，包含：

- Abstract / 摘要
- Data / 数据
- Method / 方法（LLM 抽取、聚类、AE、检索）
- Experiments / 实验设置
- Results / 结果（Recall 表、baseline 对比、消融）
- Error Analysis / 错误分析
- Limitations / 局限
- Appendix / 附录（prompt、关键词示例、Top-10 案例）

图表：

- Recall@10 对比柱状图
- 每个 query 的命中/未命中表
- AE 训练 loss 曲线
- 关键词 cluster 示例
- 典型失败案例

4. 需要你确认的 3 件事

1. **Ground Truth**：是否用 `JD + Hired CV examples` 的 10 组作为评估集？如果是，候选池是否用 100 + 10 = 110 份 CV？如果你有另一份 GT 文件，请告诉我路径。
2. **本地 LLM**：先用现有的 `qwen2.5-coder-14b-instruct` 和 `deepseek-r1-0528-qwen3-8b` 做小样本对比，再决定主力模型，可以吗？如果都不行，我再下载 `Qwen2.5-7B-Instruct`。
3. **报告形式**：一份 PDF 内中英双语分节（推荐），还是中英各一份 PDF？

你确认这三点后，我就开始在 `HW4_6207` 里复制数据、修通 LM Studio、按上面的流程执行。

用户 / User • 201

回答： 1：ground truth可以说就是JD + Hired CV examples里的标准答案，因此我认为需要纳入训练集。如果只用于评估而不用于训练，模型根本不知道如何从零开始学习正确匹配。 2：请再读一遍JD + Hired CV examples里的每个pdf，他的结构是”职位具体描述“+”一位经人工判断而符合条件的候选人“ 3：我发现有一个误区：excel表其实在当前任务下是不需要考虑的。因为我们现在要做的就是【使用”非标签化的自然语言化的“ 简历（Anonymised CVs）和职位描述(在JD + Hired CV examples文件夹里，每一份pdf都有一个职位描述和对应的1位候选人，这个职位描述是自然语言的)】 因为这项作业的根本性质就是，使用AI技术，在”职位需求和候选人简历都是非统一格式，非标签化的自然语言内容的情况下，将职位需求和候选人简历用模型编码成可以比对的关键词，然后再比对两者的相似程度（也就是候选人简历的关键词，同职位需求的关键词 的符合程度） “

助手 / Assistant • 202

明白，这版口径我完全对齐了：

- **不使用 Excel**（50 JD / 100 条 CV 摘要都不参与）
- `JD + Hired CV examples` **是核心数据**：每份 PDF 前半是自然语言 JD，后半是 `CV OF THE HIRED CANDIDATE`，即 1 组“JD + 人工认定合格候选人”
- `Anonymised-CVs-100` **是候选池和负例来源**：100 份纯自然语言 CV
- 10 份 hired CV 不在那 100 份里，所以**候选池 = 100 + 10 = 110 份 CV**

- GT **参与训练**；但为了避免“训练集=测试集”导致 Recall 虚高，主评估用 **leave-one-pair-out**，另报 in-sample 上界

一、训练与评估口径（这是关键）

你说得对：如果 10 组 GT 只用于评估，模型没有监督信号。所以我会这样做：

1. **无监督预训练 AE**：用 110 份 CV + 10 份 JD 的关键词向量做重构训练，让 AE 学会文档表示
2. **监督匹配微调**：在 AE bottleneck 后加一个 retrieval head，用 10 组 GT 做 **contrastive / listwise loss**——每个 JD 的 positive 是它的 hired CV，其余 109 份 CV 作为 negatives
3. **主评估：leave-one-pair-out（10 折）**
   - 每折留出 1 组 GT（JD + hired CV）
   - 用其余 9 组 + 100 份匿名 CV 训练
   - 用留出的 JD 在 110 份 CV 中检索 Top-10
   - 10 折平均，得到可信的 Recall
   - 这样每一组 GT 都进入过训练（9 次），同时评估是 held-out
4. **辅助：in-sample 上界**
   - 用全部 10 组训练，再在同一批 10 组上评估
   - 报告时明确标注“seen pairs / upper bound”，不作为主指标

这样既满足“GT 纳入训练”，又不会用同一批数据自评自夸。

二、数据与候选池

数据	用途	数量
`Anonymised-CVs-100`	候选 CV、AE 无监督训练、负例	100
`JD + Hired CV examples` 前半	10 个 query JD	10
`JD + Hired CV examples` 后半	10 个 positive hired CV	10
候选池合计	检索范围	**110 CV**

解析规则：在每份 PDF 中按 `CV OF THE HIRED CANDIDATE` 切分；前半为 JD，后半为 CV；去掉页眉页脚和 `Hired for:` 等元信息。

三、技术路线

Step 1：本地 LLM 关键词提取

- LM Studio 本地模型：先对比 `qwen2.5-coder-14b-instruct` 与 `deepseek-r1-0528-qwen3-8b`
- 输入：10 份 JD + 110 份 CV 的自然语言文本
- 输出 JSON：`skills / tools / domains / seniority / years / languages / location / certifications`
- 每个关键词带 `term`、`category`、`importance`、`evidence`
- temperature=0；缓存 JSONL；失败重试；人工抽查 10 份

Step 2：合并/聚类关键词

- 归一化 + 缩写/同义词映射
- 用本地 `nomic-embed-text` 对关键词做 embedding
- cosine + 层次聚类，合并相似词
- 每个 cluster 选 canonical term

- 输出词表、cluster 映射、文档–关键词矩阵
- 人工检查 top 50 clusters，防止 Java/JavaScript 这类误合并

Step 3：AE 编码

- 输入：110 CV + 10 JD 的关键词矩阵（TF-IDF 或 binary）
- 结构：$V \to 512 \to 128 \to 32/64 \to 128 \to 512 \to V$
- GELU + Dropout + BatchNorm；可选 denoising（mask 15%）
- 损失：重构损失 + 监督 retrieval loss（GT pairs）
- 输出：所有文档的 32/64 维向量
- 基线：TF-IDF cosine、BM25、Nomic embedding cosine、关键词重叠

Step 4：Top-10 检索

- 每个 GT JD 编码后，与 110 份 CV 向量算 cosine
- 返回 Top-10
- 保存排名、分数、命中的关键词证据

Step 5：Recall 评估

- 主指标：**Recall@10**（每个 JD 只有 1 份相关 CV，所以等于 hit rate）
- 辅助：Recall@5、MRR@10、平均排名
- 报告 10 折均值 + 每个 query 结果 + Wilson 95% CI
- 随机基线约 $10/110 \approx 9.1\%$，报告时作为参照
- 同时对比 AE / AE+监督 / TF-IDF / BM25 / Nomic

Step 6：错误分析

每个未命中 query 分类：

1. LLM 关键词漏抽/错抽
2. 同义词聚类错误
3. must-have 权重不足
4. seniority / location / language 不匹配
5. AE bottleneck 丢信息
6. GT 本身存在歧义

再做消融：

- 有/无监督 loss
- 有/无关键词聚类
- bottleneck 16/32/64/128
- 有/无 denoising
- LLM 关键词 vs 原始文本

Step 7：中英双语 PDF 报告

- 一份 PDF，中英双语分节
- 结构：摘要、数据、方法、实验、结果、错误分析、局限、附录
- 图表：Recall@10 对比、每 query 命中表、AE loss 曲线、关键词 cluster 示例、失败案例
- 附录：LLM prompt、Top-10 检索示例

四、目录

```
HW4_6207/
├── data/raw/              # 100 CV PDF + 10 JD+CV PDF（复制进来）
├── data/processed/        # cv_text.jsonl, jd_text.jsonl, hired_cv.jsonl
├── src/                   # parse / extract / cluster / train_ae / retrieve / evaluate / report
├── outputs/keywords/      # LLM 抽取结果
├── outputs/clusters/      # 词表与 cluster
├── outputs/models/        # AE 权重、向量
├── outputs/retrieval/     # rankings.csv
├── outputs/metrics/       # metrics.json, per_query.csv
├── report/                # HTML + figures + 最终 PDF
└── README.md
```

五、我准备先做的 Phase 0

1. 在 `HW4_6207` 建目录，复制 100 份 CV 和 10 份 JD+CV PDF
2. 解析 10 份 PDF，验证 `JOB DESCRIPTION` / `CV OF THE HIRED CANDIDATE` 切分正确
3. 修通 LM Studio API，加载 `qwen2.5-coder-14b-instruct`
4. 用 5 份文档对比 Qwen2.5-Coder-14B 和 DeepSeek-R1-Qwen3-8B 的 JSON 合规率与关键词质量
5. 跑通一条最小链路：1 个 JD → 关键词 → AE → Top-10，确认数据流无误

如果这个口径没问题，我就按上面开始执行。

用户 / User • 203

稍等，我刚刚看了一下excel表。以及我们老师在给出该数据集的时候说明的内容”Starting sheet: "Manatal Examples – JD & CV (anonymised)" has 50 JDs and 100 CVs, with no match labels yet.“

我意识到了，这个意思应该指的是，这个excel表就是”作为人才招聘集团的公司方，将原始的非关键词化的-职位描述JD和人员简历CV，整理成关键词化的标签后的结果。并且JD和CV之间尚无匹配关系“

助手 / Assistant • 204

对，这个理解更准确：

> **Excel 是人工把原始 JD/CV 整理成结构化标签后的“参考版本”，不是 JD–CV 匹配标签。** 它和 10 组 hired 的区别是：Excel 有技能/岗位等标签，但没有“谁匹配谁”；hired 有匹配关系，但没有结构化标签。

所以 Excel 不应该被丢掉，但也不应该被当成 Ground Truth。它最适合用在三个地方：

1. 验证 LLM 关键词抽取质量（silver reference）

这是 Excel 最有价值的用途：

- **100 份 CV**：Excel 的 `Skills`、`Current title`、`Role family`、`Education` 是人类整理好的标签；把 LLM 从 100 份 PDF 抽出的关键词与它对比，计算：
  - 关键词 Precision / Recall / F1（精确 + 模糊/同义词匹配）
  - Jaccard 相似度
  - 技能覆盖率

- **50 份 JD**：Excel 的 `Position`、`Role family`、`Country`、`Salary`、`Remote` 等字段，可以和 LLM 从 JD 文本抽出的 role/location/salary/seniority 对比。

这能给 Step 1 一个**量化质量指标**，而不是只靠人工感觉。它只用于验证，不参与 Recall。

2. 扩充无监督语料

- 50 份 JD 的 `Job description` 是自然语言文本，可以作为**额外的未标注 JD**，用于：
  - AE 预训练
  - 关键词聚类/词表构建
- 但**不能把 Excel 的标签直接作为 AE 输入**，否则就绕过了“自然语言 → LLM 抽取关键词”这个作业核心。
- 50 份 JD 与 10 份 GT JD 可能部分重叠；按标题/文本去重，重叠的从无监督语料中剔除，避免 leave-one-pair-out 泄漏。

3. 错误分析分层

用 Excel 的 `Role family`、`Country`、`Salary` 等字段解释：

- 哪些岗位族更容易检索错？
- 哪些地区/薪资段的 CV 更难匹配？
- 是技能抽取问题，还是领域/资历问题？

修正后的数据口径

数据	角色	是否参与训练/评估
10 份 JD + hired CV PDF	10 组匹配 GT	**监督训练 + leave-one-pair-out 评估**
100 份匿名 CV PDF	原始自然语言 CV	**AE 无监督训练 + 候选池**
10 份 hired CV	正例	**候选池 + 监督训练**
Excel 50 JD / 100 CV 标签	人工标签参考	**只做抽取验证、分层分析、可选无监督 JD 语料**

候选池仍然是 **100 + 10 = 110 份 CV**；Recall 只对 10 组 GT 计算。

训练/评估仍然是

1. AE 在无标签语料上做重构预训练（100 CV + 10 hired CV + 10 GT JD + 可选 50 Excel JD，去重）
2. 用 10 组 GT 做 supervised contrastive / listwise 微调
3. **主评估：leave-one-pair-out 10 折**（每折 9 组训练、1 组留出）
4. **辅助：in-sample 上界**（全部 10 组训练 + 同批评估，明确标注）
5. 指标：Recall@10、Recall@5、MRR、平均排名
6. Excel 标签只用于：
   - LLM 抽取的 P/R/F1
   - 错误分析分层
   - 不参与 Recall 计算

这样 Excel 既没有被浪费，也不会污染匹配 Ground Truth；核心链路仍然是：

> 原始自然语言 CV/JD → 本地 LLM 关键词 → 合并/聚类 → AE 编码 → Top-10 检索 → Recall + 错误分析

如果这个口径你认可，我就按这个开始 Phase 0：复制数据、解析 10 份 PDF、跑通 LM Studio、做小样本抽取对比。

用户 / User • 205

我有个问题，如何评估检索的召回率？我看了一下这个任务就是要有”根据JD的关键词和CV的关键词，检索符合JD的CV从而实现匹配“

系统 / System • 206

The previous response was interrupted. Continue from where you left off without repeating completed content.

助手 / Assistant • 207

接着上面 LOPO 的部分说：

如果同一批 10 组既参与训练又参与测试，Recall 一定会虚高，因为模型已经见过“这个 JD 对应这份 CV”。所以主评估不能用这种口径，而要用 **leave-one-pair-out**：

- 一共 10 折
- 每折留出 1 组 GT（JD + hired CV）
- 用其余 9 组 + 100 份匿名 CV 训练
- 用留出的 JD 在 110 份 CV 里检索 Top-10
- 看 hired CV 是否在 Top-10
- 10 折平均，得到主 Recall

这样每一组 GT 都进入过训练（9 次），但评估时用的是没见过的 JD–CV 对，指标才可信。另报一个“全部 10 组训练 + 同批评估”的 in-sample 上界，并明确标注。

具体怎么算

对每个 GT JD：

```
R(q) = { 该 JD 的 hired CV }        # 只有 1 个已知相关 CV
Top10(q) = 检索返回的前 10 份 CV
Recall@10(q) = |Top10(q) ∩ R(q)| / |R(q)|
             = 1 如果 hired CV 在 Top-10，否则 0
```

因为每个 JD 只有 1 个相关 CV，所以：

```
Recall@10 = HitRate@10
```

10 个 JD 平均：

```
Recall@10 = 命中数 / 10
```

举例：

JD	hired CV 排名	Recall@10
P01	3	1
P02	14	0
P03	1	1
…	…	…
平均		**0.8**

候选池是 110 份 CV，随机检索的 Recall@10 大约是 $10/110 \approx 9.1\%$，报告里可以作为下限参照。

除了 Recall@10，还建议报：

- **Recall@5**：更严格
- **MRR@10**：`mean(1 / rank)`，命中越靠前越高
- **平均排名**：没进 Top-10 的按 11 或 110 计
- 每个 query 的命中/未命中表

Precision@10 在这里意义不大：每个 JD 只有 1 个已知正例，即使排第 1，P@10 也只有 0.1。除非后面做人工扩充标签，否则不建议把 Precision 当主指标。

一个需要注意的问题：未标注的“好 CV”

100 份匿名 CV 里可能还有别的 CV 也符合这个 JD，只是没有标签。如果系统把“另一个好 CV”排进 Top-10，而 hired CV 没进，按严格口径会算 miss，但人工看可能是合理的。

所以报告里要写清楚：

> 这里的 Recall 是 **conservative lower bound**：只把“实际被雇佣的 CV”当作唯一已知相关项；未标注的潜在好匹配不计入。

如果你想把“符合 JD 的 CV”定义得更宽，可以加一个 **pooled manual evaluation**：

1. 对每个 JD，收集 AE、TF-IDF、BM25、Nomic 四种方法的 Top-10
2. 去重，得到每个 JD 约 20–30 份候选
3. 人工判断每份 CV 是否“符合 JD”
4. 用人工判断后的相关集合计算 Recall@10

这是信息检索里常用的 pooling 方法，能处理“一个 JD 有多个好 CV”的情况，但需要额外人工标注。10 个 JD × 20–30 份，大约 200–300 次判断，工作量可控。

建议的最终评估口径

- **主指标**：10 折 leave-one-pair-out 的 **Recall@10**
- **辅助**：Recall@5、MRR@10、平均排名
- **上界**：全部 10 组训练 + 同批评估的 in-sample Recall
- **可选**：pooled 人工判断后的 Recall@10
- **基线**：TF-IDF、BM25、Nomic embedding、关键词重叠
- **对照**：AE 重构 only vs AE + supervised contrastive

这样既回答了“关键词检索准不准”，也不会因为只有 10 组标签而把 Recall 算得虚高。

用户 / User • 208

我决定进行如下安排 一：对Manatal Examples – JD & CV (anonymised)，我认为应当完全不管。这是因为提取关键词的这一步操作，应当由本地模型LLM来提取（目前的任务已经提到过使用本地LLM，在这种情况下LLM本身是不需要训练的）。因此，只需要告诉LLM该工作背景（也就是分别提取两者的关键词），要求LLM输出对应的关键词合集，也就是JD每个职位描述的关键词分别是谁（JD来源于JD + Hired CV examples里的10个标准JD）、以及CV每份简历的关键词（包括100份匿名CV以及JD + Hired CV examples里的10份标准CV）分别是什么， 二：合并/聚类相似的关键词，以及构建自编码器模型对CV和JD进行编码，可以由非本地LLM模型的程序来操作 三：检索，同上，可以不需要LLM来操作。 其他部分同上

助手 / Assistant • 209

明白，最终口径确定：

- **Excel 完全不用**
- 本地 LLM 只做**关键词抽取**，不训练
- 抽取对象：
  - **10 份标准 JD**（来自 `JD + Hired CV examples` 前半）
  - **110 份 CV**（100 份匿名 CV + 10 份标准 hired CV）
- 关键词合并/聚类、AE 编码、检索、Recall 评估全部用非 LLM 程序完成
- 10 组 GT 仍参与训练，主评估用 leave-one-pair-out

我现在开始在 `HW4_6207` 执行 Phase 0：建目录、复制数据、解析 10 份 JD+CV PDF、检查 LM Studio。

助手 / Assistant • 372

已完成，所有工作文件都在 `HW4_6207/` 下。

最终数据与模型

- **10 份标准 JD + 10 份 hired CV**：来自 `JD + Hired CV examples`，每份 PDF 前半是 JD，后半是 `CV OF THE HIRED CANDIDATE`
- **100 份匿名 CV**：来自 `Anonymised-CVs-100`
- 候选池：**110 份 CV**
- Excel 完全未使用
- 关键词抽取：本地 LM Studio，**Qwen2.5-Coder-14B-Instruct**
- 120 份文档，平均 **24.5 个关键词/份**（范围 12–46）
- 关键词聚类：Nomic embedding + 层次聚类，**1,552 个 canonical terms**（来自 1,938 个归一化变体）
- AE：`V → 512 → 256 → 32 → 256 → 512 → V`，32 维 bottleneck
- 主评估：**leave-one-pair-out（10 折）**

主要结果

方法	Recall@10	Recall@5	MRR@10	平均排名
TF-IDF cosine	0.800	0.600	0.440	10.6
Keyword overlap	0.700	0.500	0.472	12.0
BM25	0.800	0.700	0.471	10.0

方法	Recall@10	Recall@5	MRR@10	平均排名
AE + supervised（in-sample）	1.000	1.000	1.000	1.0
AE + supervised（LOPO，主指标）	**0.800**	0.400	0.320	9.3

Bottleneck 消融（LOPO Recall@10）：16 → 0.200，32 → 0.800，64 → 0.600。

错误分析要点

- **LOPO 8/10 命中**；P04 排 13、P06 排 36。
- P04 是“差一点”：positive CV 缺 `Azure Synapse`、`PySpark`、`Azure Data Engineer Associate` 等 JD 关键词。
- P06 是口径问题：JD 是 Deployment Strategist（项目/客户/交付），hired CV 是 Data Science 背景，关键词几乎零重合。
- 其他命中但排名靠后：P02 第 9、P07 第 9、P09 第 10；AE 能把正确 CV 拉进 Top-10，但 top-1 常被领域不相关的 CV 占据。
- 同义词未完全合并仍是主要误差源：`flavour/flavor`、`key account management/client relationship management`、`B2B sales/international sales`。
- **in-sample 1.00 vs LOPO 0.80**：说明 10 组 GT 很容易被记住；报告以 LOPO 为准。

关键文件

- 中英双语报告：`HW4_6207/report/HW4_6207_Report_CN_EN.pdf`（7 页，已打开预览）
- 报告 HTML：`HW4_6207/report/report_cn_en.html`
- 指标：`HW4_6207/outputs/metrics/metrics.json`
- 每 query 结果：`HW4_6207/outputs/retrieval/per_query.csv`
- 错误分析：`HW4_6207/outputs/metrics/error_analysis.md`
- 关键词：`HW4_6207/outputs/keywords/all_keywords.jsonl`
- 聚类/词表：`HW4_6207/outputs/clusters/`
- AE 模型与向量：`HW4_6207/outputs/models/`
- 复现说明：`HW4_6207/README.md`

运行 `python src/train_eval.py --mode all --epochs 100 --bottleneck 32` 可复现实验；随机种子固定为 42。

如果老师要求，我还可以继续加一版 **AE + BM25 hybrid** 实验，或者把报告改成学校模板/加组名。

用户 / User • 373

我有一个问题：你现在是已经完成任务了吧？但是我想知道，这个任务该如何”启动“

我的意思是，就像许多exe启动程序一样，这个”进行匹配CV与JD“主程序也是有一个”从头开始启动，执行整个步骤“的启动方式吧？我想了解一下到底是怎么运作的

助手 / Assistant • 382

可以这样理解：**它不是一个编译好的 `.exe`，而是一条 Python 流水线 + 两个 `.bat` 启动器。**启动器负责“从头跑”，`run_pipeline.py` 负责按顺序调用各个步骤。

两种启动方式

1. 快速启动（用已有结果，几秒钟）

双击：

```
HW4_6207\start.bat
```

或者命令行：

```
cd D:\softs_usual\opencode_windows\stat6207_final_groupproject\HW4_6207
python run_pipeline.py
```

它做的是：

```
检查已有文件
  → 重新做错误分析
  → 重新生成中英双语 PDF 报告
```

不会重新调用 LLM，也不会重新训练 AE；适合演示、改报告文字、重新出 PDF。

2. 从头完整启动（从原始 PDF 开始，20–40 分钟）

双击：

```
HW4_6207\start_full.bat
```

或者：

```
cd D:\softs_usual\opencode_windows\stat6207_final_groupproject\HW4_6207
python run_pipeline.py --full
```

`start_full.bat` 会先：

1. `lms server start`
2. 加载 `qwen2.5-coder-14b-instruct`
3. 加载 `nomic-embed`
4. 执行 `python run_pipeline.py --full`

完整流程如下：

```
start_full.bat
  |
  ├── 1. parse_data.py
  |      10 份 JD+CV PDF → JD/CV 文本
  |      100 份匿名 CV PDF → 文本
  |      输出 data/processed/*.jsonl
  |
  ├── 2. extract_keywords.py
  |      调用本地 Qwen2.5-Coder-14B
  |      对 120 份文档逐份抽取关键词
  |      输出 outputs/keywords/all_keywords.jsonl
  |
  ├── 3. cluster_keywords.py
```

```
|     关键词归一化 + Nomic embedding + 层次聚类
|     输出 outputs/clusters/vocab.json / doc_term_tfidf.npy
|
├── 4. train_eval.py --mode all
|     TF-IDF / BM25 / 关键词重叠基线
|     AE 训练 + supervised listwise loss
|     leave-one-pair-out 10 折评估
|     输出 outputs/metrics/metrics.json / per_query.csv
|
├── 5. error_analysis.py
|     找出 miss、排名、缺失关键词
|     输出 outputs/metrics/error_analysis.md
|
```

匹配本身是怎么运作的

真正的“匹配”在第 4 步：

```
JD 文本
  → LLM 抽关键词
  → 映射到 canonical keyword 词表
  → TF-IDF 向量
  → AE encoder
  → 32 维向量
        ↕ cosine similarity
CV 文本
  → LLM 抽关键词
  → 映射到同一词表
  → TF-IDF 向量
  → AE encoder
  → 32 维向量
  → 排序取 Top-10
```

评估时对 10 个 GT JD 各检索一次，看实际 hired CV 是否在 Top-10：

```
Recall@10 = 命中数 / 10
```

主评估用 **leave-one-pair-out**：每次留 1 组 JD–CV 不参与训练，用其余 9 组 + 100 份匿名 CV 训练，再检索留出的 JD。这样 10 组 GT 都进入过训练，但评估不是同一批自评。

当前结果

- LOPO Recall@10：**0.800**
- in-sample 上界：**1.000**
- TF-IDF / BM25：**0.800**
- AE 的 MRR@10：0.320，低于 TF-IDF 的 0.440
- Bottleneck 16/32/64：**0.200 / 0.800 / 0.600**

一个重要说明

目前的“主程序”是**批量实验流水线**：它针对 10 个 Ground Truth JD 跑完整评估并生成报告。

如果你想要的是“输入一份**新的 JD**，马上输出 Top-10 CV”的交互式匹配器，那还需要再加一个 `match_one.py`：加载已训练好的 AE 和 CV 向量，对新 JD 抽关键词、编码、排序。这个我可以下一步补上。

当前关键文件：

- 启动器：`HW4_6207/start.bat`、`HW4_6207/start_full.bat`
- 主程序：`HW4_6207/run_pipeline.py`
- 报告：`HW4_6207/report/HW4_6207_Report_CN_EN.pdf`
- 指标：`HW4_6207/outputs/metrics/metrics.json`
- 说明：`HW4_6207/README.md`

用户 / User • 383

所以，所谓的完整启动，就是full.bat吧，会进行从头到尾的“加载LLM” +合并关键词+构建自编码器模型+排名检索+评估召回率?

而且我注意到作业里有一条“针对基准数据（Ground Truth）中的每个职位描述，检索出排名前 10 的简历”，应该指的就是“JD + Hired CV examples里的每个JD，都要为之检索出最相近的排名前十的CV”是吗？你检查一下你的实验有没有做到这样

助手 / Assistant • 391

是的，你的理解基本正确，但有两个关键点需要区分：

1. `start_full.bat` 确实是“从头到尾”完整启动

它执行的是：

```
start_full.bat
  → 启动 LM Studio server
  → 加载 qwen2.5-coder-14b-instruct
  → 加载 nomic-embed
  → python run_pipeline.py --full
       → 解析 10 份 JD+CV PDF 和 100 份匿名 CV PDF
       → 本地 LLM 抽取 120 份文档的关键词
       → 关键词归一化 + 聚类
       → 构建 AE 并训练
       → 对每个 GT JD 检索 Top-10 CV
       → 计算 Recall@10 / Recall@5 / MRR
       → 错误分析
       → 生成中英双语 PDF 报告
```

注意：**LLM 只是被加载和调用，不训练 LLM；训练的是 AE。**
`start.bat` 是快速模式，不重新调用 LLM、不重新训练，只复用已有结果并重新生成报告。

2. “每个 JD 检索 Top-10”确实做到了

我检查了 `outputs/retrieval/per_query.csv`，LOPO 结果正好是 **10 个 JD × 每个 10 份 CV**，每行 10 个 CV ID 且无重复。

JD	hired CV 排名	是否进 Top-10
P01	2	✓
P02	9	✓
P03	1	✓
P04	13	✗

JD	hired CV 排名	是否进 Top-10
P05	8	✓
P06	36	✗
P07	9	✓
P08	4	✓
P09	10	✓
P10	1	✓

所以：

```
Recall@10 = 8 / 10 = 0.800
```

- **P04、P06 不是没有检索 Top-10**，而是它们的 hired CV 排在 13 和 36，没进前 10。
- 其他 8 个 JD 的 hired CV 都进了 Top-10。
- 候选池是 **110 份 CV**（100 匿名 + 10 hired），每个 JD 都在这 110 份里排序取前 10。

3. 报告里已经补上完整明细

原来的报告表格只显示了每个 JD 的 Top-1，容易让人以为只做了一层排序。我已新增 **附录 C**，列出每个 JD 实际返回的完整 Top-10：

`HW4_6207/report/HW4_6207_Report_CN_EN.pdf`

现在报告是 8 页，附录 C 的表格就是：

```
Pair | JD | Hired CV rank | Top-10 CVs (1 → 10)
```

例如 P01：

```
1. CV-082
2. HIRED_P01 ✓
3. CV-044
...
10. CV-015
```

P04 和 P06 的表格里没有 ✓，因为它们对应的 hired CV 排名 > 10；但 Top-10 列表本身是完整返回的。

4. 结论

- `start_full.bat`：完整从零启动，包含加载 LLM、关键词抽取、聚类、AE 训练、Top-10 检索、Recall 评估、报告生成。
- 实验确实对 `JD + Hired CV examples` 中 **全部 10 个 JD** 都做了 Top-10 检索。
- 主评估是 leave-one-pair-out；每个 JD 的 known positive 只有 1 份 hired CV，所以 Recall@10 就是“hired CV 是否进 Top-10”的命中率。
- 当前结果：**8/10 命中，Recall@10 = 0.800**。

我还有个问题：JD-P01是什么？实际上每一个“JD”都有一个正式的名称吧，你直接丢给我JD-P01我完全不知道是啥，写进报告里也显得突兀。你应当给予正确的名称

助手 / Assistant • 394

你说得对，`JD_P01` 只是内部 ID，不应该单独出现在报告里。我会把报告里的所有 JD 都改成正式职位名称，并重新生成。

助手 / Assistant • 423

你说得对，`JD_P01` 只是我内部用的文档 ID，不是正式职位名称。报告里直接写它会很突兀。现在已经全部改成正式职位名，并重新生成了报告。

10 个 JD 的正式名称

内部 ID	正式职位名称
P01	Regional Finance Controller
P02	Sales Manager in Indonesia
P03	GTM Marketing Manager / Lead
P04	Data Engineer
P05	Product Manager — WMS & Warehouse Integrations
P06	Deployment Strategist
P07	Marketing Manager (HK & SG)
P08	Field Service Engineer (FSE)
P09	Tech Lead
P10	Regional Lead Designer UI/UX

报告已改的地方

- 结果表：从 `JD_P01` 改成 **P01 + Regional Finance Controller**
- 错误分析表：改成 **P04 + Data Engineer**、**P06 + Deployment Strategist**
- 附录 C 的完整 Top-10 表：每个 JD 都显示正式职位名
- `outputs/retrieval/per_query.csv` 新增 `role_title` 列
- `outputs/metrics/error_analysis.md` 的标题也改成正式名称
- 报告正文里的 P02/P07/P08/P09 也都补上了职位名

现在报告里已经搜不到 `JD_P01` 这类内部 ID，只保留 `P01` 作为序号，正式名称紧随其后。

顺便再确认你问的 Top-10

确实是对 `JD + Hired CV examples` 里的**每一个 JD** 都检索了 Top-10，共 10 个 JD × 10 份 CV：

JD	正式名称	hired CV 排名	是否进 Top-10
P01	Regional Finance Controller	2	✓
P02	Sales Manager in Indonesia	9	✓
P03	GTM Marketing Manager / Lead	1	✓

JD	正式名称	hired CV 排名	是否进 Top-10
P04	Data Engineer	13	✗
P05	Product Manager — WMS & Warehouse Integrations	8	✓
P06	Deployment Strategist	36	✗
P07	Marketing Manager (HK & SG)	9	✓
P08	Field Service Engineer (FSE)	4	✓
P09	Tech Lead	10	✓
P10	Regional Lead Designer UI/UX	1	✓

`P04` 和 `P06` 不是没有返回 Top-10，而是它们的 hired CV 排名 13 和 36，没进前 10，所以算 miss。最终 **Recall@10 = 8/10 = 0.800**。

更新后的报告：

`HW4_6207/report/HW4_6207_Report_CN_EN.pdf`

现在 8 页，附录 C 可以看到每个 JD 实际返回的完整 Top-10 列表。如果你希望连 `P01` 这个序号也去掉、只保留职位名称，我也可以再改一版。

用户 / User • 424

关于报告的生成方式：我认为应当中文英文各一份也就是共两份报告，而不是中英文混搭为同一份报告 此外，我好奇你是如何生成报告的？光凭本地模型是不可能做到的吧

助手 / Assistant • 425

明白，两个要求：

1. **报告拆成中文、英文各一份**，不再中英混排。
2. 我会说明报告到底是怎么生成的——**不是本地模型写报告**，本地模型只负责关键词抽取。

我先把报告生成器改成双份输出，再解释技术链路。

助手 / Assistant • 444

已经改成**中文、英文各一份**，不再中英混排。两份都是 7 页：

- 中文报告：`HW4_6207/report/HW4_6207_Report_CN.pdf`
- 英文报告：`HW4_6207/report/HW4_6207_Report_EN.pdf`

旧的合并版 `HW4_6207_Report_CN_EN.pdf` 已删除。`start.bat` 和 `start_full.bat` 现在都会同时生成这两份。

报告到底是怎么生成的

你的直觉是对的：**光靠本地 LLM 不可能生成这种带表格、图表、排版的 PDF。**本地 LLM 在这个项目里只做一件事：**从 JD/CV 文本中抽取关键词。**

报告的生成链路是：

```
本地 LLM（Qwen2.5-Coder-14B）
  → 关键词 JSONL
  → Python 聚类 / AE / 检索 / 评估
  → metrics.json + per_query.csv + error_analysis.json + keyword_clusters.json
  → Python 报告模板（中文一份、英文一份）
  → matplotlib 生成图表
  → HTML + CSS
  → Microsoft Edge headless --print-to-pdf
  → 中文 PDF / 英文 PDF
```

具体来说：

1. **本地 LLM 不写报告**
    - 只负责 `extract_keywords.py` 里的关键词抽取。
    - 报告正文不是模型生成的，而是 `src/make_report.py` 里预先写好的中文/英文模板。
2. **数据由程序计算**
    - Recall、MRR、平均排名来自 `outputs/metrics/metrics.json`。
    - 每个 JD 的 Top-10 来自 `outputs/retrieval/per_query.csv`。
    - 错误分析来自 `outputs/metrics/error_analysis.json`。
    - 关键词聚类来自 `outputs/clusters/keyword_clusters.json`。
3. **图表由 matplotlib 生成**
    - 中文报告用中文字体（Microsoft YaHei）生成中文图表。
    - 英文报告单独生成英文图表。
4. **PDF 由浏览器打印**
    - 报告先写成 HTML/CSS。
    - 再用 **Microsoft Edge headless** 的 `--print-to-pdf` 转成 A4 PDF。
    - 这也是为什么排版、表格、分页、图注能做得比较整齐，而不是模型直接“写”出来的。

所以整条链路是：

> 本地 LLM 负责“读懂文本、抽关键词”；Python 负责“计算、画图、填模板”；Edge 负责“排版成 PDF”。

中文版和英文版的区别

- 中文版正文全部中文，只保留 `LLM`、`Recall@10`、`AE` 这类技术术语，以及官方英文职位名。
- 英文版全部英文。
- 两份报告的表格、图、指标、Top-10 明细完全一致，只是语言不同。
- 每个 JD 都显示正式职位名，例如：
    - P04 → Data Engineer
    - P06 → Deployment Strategist
    - P10 → Regional Lead Designer UI/UX

启动方式

- 快速生成两份报告（复用已有结果）：

```
HW4_6207\start.bat
```

- 从头完整运行（加载 LLM、抽关键词、聚类、训练 AE、检索、评估、生成两份报告）：

```
HW4_6207\start_full.bat
```

- 直接命令行：

```
python run_pipeline.py
python run_pipeline.py --full
```

`run_pipeline.py` 最后的输出会分别指向：

```
report (zh): HW4_6207\report\HW4_6207_Report_CN.pdf
report (en): HW4_6207\report\HW4_6207_Report_EN.pdf
```

用户 / User • 445

我想知道一下，也就是生成报告不需要连接像你这样的大模型？因为你已经编排好了格式，所以后续生成报告只是在已有内容上修改数据内容吗？

助手 / Assistant • 446

对，**生成报告阶段完全不需要连接任何大模型**，也不需要调用我。整个报告生成是本地、确定性、离线的。

但准确说，不是“所有内容都自动改数据”，而是分两类：

1. 动态部分：每次从结果文件读取

这些会随实验数据自动更新：

- 指标表：Recall@10、Recall@5、MRR、平均排名、95% CI
- 每个 JD 的 Top-10 表
- 错误分析表
- Bottleneck 消融表
- 三张图（Recall 柱状图、LOPO 排名图、关键词聚类图）
- “主要结果”框里的数字

数据来源：

```
outputs/metrics/metrics.json
outputs/retrieval/per_query.csv
outputs/metrics/error_analysis.json
outputs/clusters/keyword_clusters.json
```

2. 静态部分：文字分析已经写好

报告里的分析段落、改进建议、局限说明，是提前写在 `src/make_report.py` 里的中文/英文模板，例如：

- “AE 与基线打平 Recall，但 MRR 更低”
- “P04 是 near miss，P06 关键词几乎零重合”
- “建议加入 must-have 加权和 hybrid 检索”

这些**不会**在每次运行时由模型重新生成。

其中有些句子里的数字目前是写死的，比如“0.80” “0.32” “P04 rank 13” 。如果以后换数据或模型，表格和图会自动变，但这些叙述句可能仍然是旧数字，需要同步改模板，否则会出现“表里是 0.9、文字还写 0.8”的不一致。

报告生成的真实链路

```
metrics.json / per_query.csv / error_analysis.json / keyword_clusters.json
        ↓
make_report.py 里的中文模板 + 英文模板
        ↓
matplotlib 画图（中文图 / 英文图）
        ↓
HTML + CSS
        ↓
Microsoft Edge headless --print-to-pdf
        ↓
HW4_6207_Report_CN.pdf
HW4_6207_Report_EN.pdf
```

全程只用到：

- Python
- matplotlib
- HTML/CSS
- 本地 Edge 浏览器

不需要 LM Studio，不需要 Qwen，不需要云端 API，也不需要我。

和 `start.bat` / `start_full.bat` 的关系

- `start.bat` 快速模式：**完全不调用 LLM**，只做错误分析 + 重新生成两份 PDF。
- `start_full.bat` 完整模式：本地 Qwen 只负责**关键词抽取**；后面的聚类、AE、检索、报告仍然由 Python 完成。

所以你说的“格式已经编排好，后续只是填数据”大体正确，但更准确的说法是：

> **表格、图、关键指标是自动填数据；分析文字是固定模板，部分数字需要人工同步。**

如果你希望，我可以再改一版，把分析段落也做成**完全参数化**的：所有数字都从 `metrics.json` 读取，根据命中/未命中自动选择句子，这样换数据后报告文字也不会和数字脱节。

用户 / User • 447

不必了。总之我只需要知道，我点击start_full.bat就能进行完整模式吧。

还有一个问题：D:\softs_usual\opencode_windows\stat6207_final_groupproject\HW4_6207\data\raw 是不是原始数据文件夹？

咋说呢，我现在的思路转变了，变为“写成一个可用软件“，具体来说就是让老师或者我自己拿到后就能马上用。 所以，我是否可以理解为，raw文件夹下的cvs100，是原始的CV文件夹（本地LLM将从这里读取与CV有关的信息并提取关键词）；而jd_cv_pairs，是原始的JD文件夹（本地LLM将从这里读取与JD有关的信息并提取关键词）？ 只要使用者分别往两个文件夹，丢入任意的CV和JD，然后跑一遍主程序，就能产出一份”检索列表“？现在能实现这样的效果吗？如果不能，你打算如何改进优化

助手 / Assistant • 448

先回答你的两个判断：

1. 点击 `start_full.bat` 能跑完整模式吗？

能，但它跑的是“当前 benchmark 数据”的完整流程，不是通用匹配软件。

`start_full.bat` 会：

```
启动 LM Studio
→ 加载 Qwen + Nomic
→ 解析 data/raw 下的 PDF
→ LLM 抽关键词
→ 关键词聚类
→ 训练 AE
→ 对 10 个 GT JD 检索 Top-10
→ 计算 Recall
→ 生成中文/英文报告
```

如果换一台机器，前提是：Python 依赖、LM Studio、两个本地模型、Edge 都已安装。它不是打包好的 `.exe`。

2. `data/raw` 是原始数据文件夹吗？

是。但两个子文件夹的性质不同：

```
data/raw/
├── cvs_100/          # 100 份纯 CV PDF
└── jd_cv_pairs/      # 10 份“JD + hired CV”合体 PDF
```

- `cvs_100`：纯简历 PDF。LLM 从这里读取 CV 内容并抽关键词。
- `jd_cv_pairs`：**不是纯 JD 文件夹。**每份 PDF 前半是 JD，后半是 `CV OF THE HIRED CANDIDATE`。`parse_data.py` 会在 `CV OF THE HIRED CANDIDATE` 处切分：
  - 前半 → JD
  - 后半 → hired CV（作为该 JD 的 known positive）

所以它同时承担了“JD 来源”和“Ground Truth 正例来源”两个角色。

---

3. 任意丢 CV/JD 进去，就能出检索列表吗？

目前还不能完全做到。 现在的程序是“实验/评测流水线”，不是通用匹配工具。具体限制：

能做的

- 往 `cvs_100` 里放任意 CV PDF，`parse_cvs()` 会逐个读取，用文件名当 `doc_id`。

- 往 `jd_cv_pairs` 里放 **同样结构的“JD + hired CV”PDF**，可以重新抽关键词、重新训练、重新评估。

不能直接做的

1. **纯 JD PDF 丢进 `jd_cv_pairs` 会报错**
   - `parse_data.py` 要求每份 PDF 里出现 `CV OF THE HIRED CANDIDATE`。
   - 如果只是普通 JD，没有这个标记，会直接 `ValueError: Marker not found`。
2. **Ground Truth 是固定文件**
   - `train_eval.py` 读 `data/processed/ground_truth.json`，里面固定是 10 组 `(jd_id, cv_id)`。
   - 如果你换成别的 JD/CV，这个文件不会自动更新，评估会失败或算错。
3. **没有“输入一份新 JD → 输出 Top-10”的查询模式**
   - 现在是对固定的 10 个 GT JD 批量评测。
   - 没有 `match.py --jd xxx.pdf` 这种单条查询入口。
4. **报告是评测报告**
   - 现在的报告包含 Recall、LOPO、错误分析。
   - 如果没有 Ground Truth，就没有 Recall 可算，报告结构也不适用。

---

4. 要变成“可用软件”，我打算这样改

目标使用方式：

```
HW4_6207/
├── data/input/
|   ├── cvs/           # 任意 CV PDF / TXT / DOCX
|   └── jds/           # 任意 JD PDF / TXT / DOCX
├── data/ground_truth/
|   └── ground_truth.csv   # 可选；没有就跳过评估
├── index/            # 构建好的词表、聚类、AE、CV 向量
├── outputs/
|   ├── rankings.csv   # 每个 JD 的 Top-10 CV
|   └── rankings.md
├── report/
|   ├── report_CN.pdf
|   └── report_EN.pdf
├── start_full.bat    # 建索引 + 全量匹配 + 报告
└── match_one.bat     # 拖入一份 JD，直接出 Top-10
```

改造分四步：

第一步：拆分“输入”和“评测”

- 新增 `data/input/jds/`：**纯 JD**，不再要求包含 hired CV。
- 新增 `data/input/cvs/`：任意 CV。
- 保留 `data/raw/jd_cv_pairs/` 作为旧 benchmark 数据，不再作为通用输入。
- `parse_data.py` 改成：

    - 普通 JD → 直接解析为 JD
    - 旧 benchmark PDF → 自动检测 `CV OF THE HIRED CANDIDATE` 并切分
    - 文件名 → `doc_id`，不再依赖 `P01` 命名

第二步：把“建索引”和“查询”分开

新增：

- `build_index.py`
    - 解析所有 CV/JD
    - 调用本地 LLM 抽关键词
    - 聚类、建词表
    - 训练 AE
    - 保存 `index/`（词表、聚类、AE 权重、CV 向量）
- `match.py`
    - 加载 `index/`
    - 输入一份 JD（或整个 `jds/` 文件夹）
    - 抽关键词 → 映射到已有词表 → AE 编码 → 与所有 CV 向量算 cosine
    - 输出 Top-10：`rankings.csv`、`rankings.md`

这样：

- **CV 集合变了** → 重新跑 `build_index.py`
- **只新增 JD** → 不用重训 AE，直接跑 `match.py`

第三步：Ground Truth 变成可选

- 有 `ground_truth.csv` → 计算 Recall@10、MRR，生成评测报告。
- 没有 → 跳过评估，只生成“匹配列表报告”，不虚构 Recall。

第四步：启动器

- `start_full.bat`：建索引 + 全量匹配 + 可选评估 + 中英文报告
- `match_one.bat`：把一份 JD 拖上去，直接输出 Top-10
- `setup.bat`：检查 Python、LM Studio、模型、Edge 是否齐全

结论

- `start_full.bat` 现在是**完整 benchmark 流程**，可以跑通当前 10 组数据。
- `data/raw/cvs_100` 是纯 CV 来源。
- `data/raw/jd_cv_pairs` 是 **JD + hired CV 合体**，不是纯 JD 文件夹。
- “任意丢 CV/JD 进去就出检索列表”**目前还不能**，主要卡在 JD 解析格式、固定 Ground Truth、缺少单 JD 查询模式，以及报告是评测报告。
- 改造成本不高，核心是：**输入目录标准化 + 建索引/查询分离 + GT 可选 + 单 JD 匹配入口。**

如果你要，我下一步就直接按这个结构改：新增 `data/input/cvs`、`data/input/jds`、`build_index.py`、`match.py`、`match_one.bat`，并把 `start_full.bat` 改成通用流程。

用户 / User • 449

parse_data.py 要求每份 PDF 里出现 CV OF THE HIRED CANDIDATE。 这个是什么？我不太了解？ 总之我觉得，要知道本地LLM也是LLM，她应该存在”即使输入的文件不具有规范的格式，也能凭借LLM自身的理解自然语言能力来进行提取 “。

助手 / Assistant • 450

`CV OF THE HIRED CANDIDATE` 不是通用格式要求，而是这 10 份 benchmark PDF 里的一个**分页标题/分隔标记。**

1. 它到底是什么

以 `P01 - Regional Finance Controller.pdf` 为例，PDF 内容结构是：

```
第 1–3 页：
P01 • JOB DESCRIPTION • Finance & Legal
Regional Finance Controller
Kuala Lumpur, Malaysia • full time • MYR 25,000 / month
Position Summary...
Responsibilities...
Experience...
Qualifications...

第 4 页开始：
P01 • CV OF THE HIRED CANDIDATE
Regional Finance Manager
Cardia Bioplastics (Malaysia) Sdn Bhd • Malaysia
Hired for: Regional Finance Controller
Skills...
Work experience...
Education...
```

所以 `CV OF THE HIRED CANDIDATE` 的字面意思是：

> **“这个岗位最终被录用候选人的简历”**

它是 Second Talent 在制作 benchmark 时加进去的标注，用来把同一份 PDF 分成两半：

```
前半部分 = JD
后半部分 = 被录用者的 CV（Ground Truth 正例）
```

`parse_data.py` 里的核心代码就是：

```
jd_raw, cv_raw = text.split("CV OF THE HIRED CANDIDATE", 1)
```

它不是在要求“所有 JD 都必须长这样”，而是在处理这 10 份**JD + hired CV 合体 PDF**时，找一个可靠的分割点。

2. 你说得对：限制不在 LLM，而在预处理

本地 LLM 确实能处理不规范的文本。只要把自然语言文本喂给它，它就可以抽取技能、工具、领域、资历等关键词，不要求固定标题。

但当前流程有两层：

```
PDF 文件
  → PyMuPDF 提取文本（格式/分页/表格）
  → parse_data.py 切分 JD / CV（当前依赖 marker）
  → LLM 抽关键词（不依赖格式）
  → 聚类 / AE / 检索
```

问题出在**第二步**，不是 LLM。

- 如果 `jd_cv_pairs` 里是一份纯 JD PDF，没有 `CV OF THE HIRED CANDIDATE`，`parse_data.py` 会直接报错。
- 但如果你把这份纯 JD 的文本直接给 LLM，它完全可以抽关键词。
- 所以“LLM 能理解非规范格式”是对的；“当前代码能接收任意格式”是不对的。

3. 为什么当初要这个 marker

因为这 10 份 PDF 不是“一个文件一份 JD”或“一个文件一份 CV”，而是：

```
1 个 PDF = 1 个 JD + 1 个 hired CV
```

要同时得到：

- JD 文本 → 作为 query
- hired CV 文本 → 作为 known positive / Ground Truth

就必须知道从哪里切开。marker 是最便宜、最稳定的切法。

如果是一般的软件输入，更合理的做法是**一开始就分开**：

```
data/input/jds/     # 一份文件 = 一个 JD
data/input/cvs/     # 一份文件 = 一个 CV
```

这样根本不需要 marker，LLM 直接对每个文件抽关键词。

4. 通用化应该怎么改

方案 A：推荐，输入时就分开

```
data/input/
├── jds/
|   ├── Regional Finance Controller.pdf
|   ├── Data Engineer.pdf
|   └── ...
└── cvs/
    ├── 张三.pdf
    ├── 李四.pdf
    └── ...
```

流程：

```
每个 JD 文件 → 提取文本 → LLM 抽 JD 关键词
每个 CV 文件 → 提取文本 → LLM 抽 CV 关键词
→ 聚类 / AE / 检索
```

不需要任何 `CV OF THE HIRED CANDIDATE` 标记。

方案 B：保留 benchmark 兼容

- `data/raw/jd_cv_pairs/` 继续走 marker 切分，用于复现当前 10 组实验。
- 新增 `data/input/jds/` 和 `data/input/cvs/` 作为通用模式。
- `parse_data.py` 自动判断：
  - 有 marker → 按 benchmark pair 处理
  - 没有 marker → 当作纯 JD 或纯 CV 处理

方案 C：LLM 辅助切分

如果用户就是丢进来一份“JD + CV 混在一起”的 PDF，可以让 LLM 做一次分段：

```
请把下面的文本分成两部分：
1. job_description
2. candidate_cv
返回 JSON：{"jd_text": "...", "cv_text": "..."}
```

LLM 可以靠语义判断“Responsibilities/Requirements”是 JD，“Work experience/Education”是 CV。但这个方式有两个问题：

- 多一次 LLM 调用，成本和延迟增加；
- 对格式特别怪的文件，分段可能不稳定。

所以生产环境更推荐方案 A：**输入时就把 JD 和 CV 分文件夹。**

5. 还有一个现实限制

LLM 再强，也只能读**文本**。如果 PDF 是扫描件/图片，没有文本层，PyMuPDF 提取不到文字，LLM 也看不到内容。这种情况需要 OCR，或者用支持视觉的模型。当前流程默认 PDF 有文本层。

结论

- `CV OF THE HIRED CANDIDATE` = benchmark PDF 里“被录用候选人简历”的分页标记。
- 当前代码需要它，是因为这些 PDF 把 JD 和 CV 合在同一个文件里，需要切分。
- LLM 本身不要求这种格式；限制来自 `parse_data.py` 的预处理规则。
- 要做成通用软件，应该把输入改成：
  - `jds/` 一份文件一个 JD
  - `cvs/` 一份文件一个 CV
  - 不再依赖 marker
- 可选保留 benchmark 模式，兼容现有 10 组数据。

如果你要，我下一步就把 `parse_data.py` 改成“通用输入 + benchmark 兼容”两种模式，并新增 `data/input/jds`、`data/input/cvs`。

用户 / User • 451

我觉得进行方案A就行了，也就是具备通用化功能

助手 / Assistant • 453

好，我按方案 A 做：新增通用输入目录和通用流水线，保留原来的 benchmark 模式。先改成通用版，再用小样本跑通验证。

助手 / Assistant • 494

对，`start_full.bat` 是当前 benchmark 的完整模式。但如果要“任意 CV/JD 丢进去就能用”，请用新加的 `start_general.bat`。方案 A 已经实现并跑通验证。

先确认原始数据

```
data/raw/
├── cvs_100/          # 100 份纯 CV PDF
└── jd_cv_pairs/      # 10 份“JD + hired CV”合体 PDF
```

- `cvs_100` 是原始 CV。
- `jd_cv_pairs` 不是纯 JD，它每份 PDF 同时包含 JD 和 `CV OF THE HIRED CANDIDATE`；这是 benchmark 的 ground truth 来源。
- 通用模式不再使用这个结构，改用下面两个干净文件夹。

通用模式怎么用

1. 放数据

```
HW4_6207/data/input/
├── jds/          # 一份文件 = 一个 JD
|   ├── Data Engineer.pdf
|   └── Product Manager.txt
└── cvs/          # 一份文件 = 一份 CV
    ├── candidate_a.pdf
    └── candidate_b.docx
```

支持 `.pdf`、`.txt`、`.md`、`.docx`。
不需要 `CV OF THE HIRED CANDIDATE`，因为 JD 和 CV 已经分文件夹了。

2. 启动

双击：

```
HW4_6207\start_general.bat
```

或命令行：

```
cd HW4_6207
python run_general.py
```

它会执行：

```
解析 data/input/jds 和 data/input/cvs
→ 本地 Qwen 抽关键词
```

```
→ 关键词聚类
→ 训练 AE
→ 对每个 JD 检索 Top-10 CV
→ 输出检索列表
```

3. 输出

```
outputs/general/rankings.csv
outputs/general/rankings.md
```

- `rankings.csv`：长表，包含 `jd_id, jd_title, rank, cv_id, cv_title, score, shared_keywords`
- `rankings.md`：每个 JD 一段可读的 Top-10 表格

示例格式：

```
## Data Engineer

| Rank | CV | Score | Shared keywords |
|---:|---|---:|---|
| 1 | CV-001 | 0.87 | python, sql, spark |
| 2 | CV-003 | 0.81 | python, etl |
...
```

4. 可选：计算 Recall

如果你有 Ground Truth，在：

```
data/input/ground_truth.csv
```

写：

```
jd_id,cv_id
JD_Data_Engineer,CV_CV-001
```

或者用文件名：

```
jd_file,cv_file
Data Engineer.pdf,CV-001.pdf
```

有它才会输出：

```
outputs/general/metrics.json
```

没有它也能正常出检索列表，只是不计算 Recall。

三个启动器的区别

启动器	输入	用途
`start.bat`	已有 benchmark 缓存	几秒内重新生成中文/英文报告
`start_full.bat`	`data/raw` 的 10 组 benchmark	完整复现实验：抽关键词、训练 AE、Recall、报告
`start_general.bat`	`data/input/jds` + `data/input/cvs`	通用匹配：任意 JD/CV → Top-10 列表

所以：

- 想复现作业实验 → `start_full.bat`
- 想拿自己的 CV/JD 直接用 → `start_general.bat`

现在的限制

通用模式已经是可用的，但还有几个边界：

1. **每次都会重建索引**
    - 换了 CV 集合，必须重新抽关键词、聚类、训练 AE。
    - 目前还没有“只加一份新 JD、直接查已有索引”的轻量查询模式。
2. **扫描版 PDF 不支持**
    - 目前依赖 PyMuPDF 提取文本层；纯图片 PDF 需要 OCR。
    - 文本型 PDF、TXT、DOCX 没问题。
3. **没有 Ground Truth 时只出列表，不出 Recall**
    - 这是合理的：没有标签就没有 Recall 可算。
4. **本地 LLM 依赖**
    - `start_general.bat` 仍然需要 LM Studio、Qwen 和 Nomic。

如果你希望更接近“软件”，下一步可以再加一个：

```
match_one.bat
```

把一份 JD 拖上去，直接输出它的 Top-10，不用重建整个索引。这个我可以继续加。

用户 / User • 495

总之现在应该够了。你能否在readme或者“使用手册”里，增加中英文的“使用说明”内容？让老师在得到整个项目文件后可以复现。

助手 / Assistant • 505

已经加好了，放在项目根目录：

- `HW4_6207/使用手册_User_Manual.md`
- `HW4_6207/README.md` 顶部也加了指向这份手册的链接

使用手册包含的内容

中英文双语，按这个顺序：

1. **项目是什么**
    - benchmark 复现模式

- general 通用匹配模式

2. **项目结构**
   - `data/raw`、`data/input`、`outputs`、`report` 各自是什么
3. **环境准备**
   - Python 依赖：`pip install -r requirements.txt`
   - LM Studio 安装与两个模型加载命令
   - API 检查：`http://127.0.0.1:1234/v1/models`
   - Edge 路径说明
4. **模式 A：复现实验**
   - 双击 `start_full.bat`
   - 或 `python run_pipeline.py --full`
   - 运行步骤、预计时间、输出文件
   - 预期指标表：Recall@10、MRR、Bottleneck 消融
5. **模式 B：通用匹配**
   - 把 JD 放进 `data/input/jds`
   - 把 CV 放进 `data/input/cvs`
   - 支持 PDF/TXT/MD/DOCX
   - 双击 `start_general.bat`
   - 输出 `outputs/general/rankings.csv` 和 `rankings.md`
   - 可选 `ground_truth.csv` 计算 Recall
6. **三个启动器的区别**
   - `start.bat`：快速重出报告
   - `start_full.bat`：完整 benchmark 复现
   - `start_general.bat`：任意 JD/CV 通用匹配
7. **报告生成方式**
   - 本地 LLM 只抽关键词
   - Python + matplotlib + HTML/CSS + Edge 生成 PDF
8. **复现性**
   - seed 42、`config.json`、缓存机制
9. **常见问题**
   - LM Studio API 连不上
   - 模型没加载
   - Python 包缺失
   - Edge 找不到
   - `Marker not found`
   - 扫描版 PDF 没文本
   - 运行很慢

10. **最短复现 checklist**
    - 老师拿到项目后按 1–6 步即可跑通

老师拿到项目后的最短流程

```
1. 打开 HW4_6207
2. pip install -r requirements.txt
3. 启动 LM Studio，加载 qwen2.5-coder-14b-instruct 和 nomic-embed
4. 双击 start_full.bat（复现实验）
   或 start_general.bat（用自己的 JD/CV）
5. 查看 report/HW4_6207_Report_CN.pdf 和 Report_EN.pdf
```

如果老师不方便看 Markdown，我也可以把这份使用手册再导出成一份中英双语的 PDF。

用户 / User • 506

接下来，我们老师要求我们提交以下文件：报告pdf、以及完整会话记录（非概括性内容，是要完整的详细会话记录，不需要你给我概括），帮我整理一份吧

助手 / Assistant • 521

我会从 OpenCode 的会话数据库里导出**完整对话原文**（不是摘要），并把工具调用记录作为附录保留；同时把两份报告复制到提交目录。